

LAPORAN PRAKTIKUM STRUKTUR DATA
IMPLEMENTASI DAN PENERAPAN STRUKTUR DATA STACK PADA
BAHASA PEMROGRAMAN JAVA



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Tuhan Yang Maha Esa karena rahmat dan karunianya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah yang berkaitan dengan struktur data.

Laporan ini membahas mengenai implementasi struktur data stack menggunakan bahasa pemrograman java. Dalam praktikum ini, dilakukan beberapa percobaan seperti pembuatan stack menggunakan array, penggunaan stack bawaan java, evaluasi ekspresi postfix, pencarian nilai maksimum dalam stack, serta penerapan stack dalam pengelolaan data objek. Melalui kegiatan ini, diharapkan dapat memahami konsep kerja stack serta penerapannya dalam berbagai kasus.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa yang akan datang.

Padang, 14 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Dasar Teori	6
2.2 Langkah Kerja.....	7
2.2.1 Kode StackArray_2511532024.....	7
2.2.2 Penjelasan Kode StackArray_2511532024	8
2.2.3 Kode StackArrayDriver_2511532024	10
2.2.4 Penjelasan Kode StackArrayDriver_2511532024.....	10
2.2.5 Kode StackPostfix_2511532024	12
2.2.6 Penjelasan Kode StackPostfix_2511532024.....	13
2.2.7 Kode NilaiMaksimum_2511532024.....	16
2.2.8 Penjelasan Kode NilaiMaksimum_2511532024	16
2.2.9 Kode SiswaStack_2511532024	19
2.2.10 Penjelasan Kode SiswaStack_2511532024.....	20
2.2.11 Output Kode Program.....	25
BAB III PENUTUP	26
3.1 Kesimpulan	26
DAFTAR PUSTAKA.....	27

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data yang efisien merupakan hal yang sangat penting. Salah satu konsep dasar yang digunakan dalam pengelolaan data adalah struktur data. Struktur data berfungsi untuk mengatur dan menyimpan data agar dapat diakses dan dimanipulasi dengan lebih mudah dan terstruktur.

Salah satu jenis struktur data yang sering digunakan adalah stack. Stack merupakan struktur data yang bekerja dengan prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Konsep ini banyak digunakan dalam berbagai aplikasi, seperti pengolahan ekspresi matematika, manajemen memori, serta proses undo dan redo pada aplikasi.

Pada praktikum ini, implementasi struktur data stack dilakukan menggunakan beberapa pendekatan, yaitu menggunakan array, menggunakan kelas stack bawaan java, serta penerapannya dalam berbagai kasus seperti evaluasi ekspresi postfix, pencaarian nilai maksimum, dan pengelolaan data objek. Dengan mempelajari berbagai implementasi tersebut, diharapkan dapat memberikan pemahaman yang lebih mendalam mengenai cara kerja dan penerapan struktur data stack dalam pemrograman.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data stack dan prinsip kerjanya (Last In First Out).
2. Mampu mengimplementasikan stack menggunakan array dan stack bawaan java.
3. Mampu menerapkan stack dalam berbagai kasus, seperti evaluasi ekspresi postfix dan pencarian nilai maksimum.

4. Meningkatkan kemampuan dalam mengelola data menggunakan struktur data yang tepat.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Mahasiswa dapat memahami penggunaan stack secara praktis dalam pemrograman.
2. Mahasiswa mampu mengembangkan logika pemrograman dalam menyelesaikan permasalahan berbasis struktur data.
3. Membantu dalam memahami penerapan stack dalam kasus nyata didunia pemrograman.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Struktur data stack merupakan salah satu struktur data linier yang digunakan untuk menyimpan data dengan urutan tertentu. Berdasarkan materi praktikum, stack bekerja dengan prinsip Last In first Out (LIFO), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Konsep ini juga dikenal dengan istilah First In Last Out (FILO), yang menunjukkan bahwa elemen pertama yang masuk akan menjadi elemen terakhir yang keluar.

Secara umum, stack memiliki dua jenis, yaitu stack dengan ukuran tetap dan stack dengan ukuran dinamis. Stack ukuran tetap memiliki kapasitas yang tidak dapat berubah, sehingga jika penuh akan terjadi kondisi overflow, dan jika kosong lalu dilakukan penghapusan akan terjadi underflow. Sedangkan stack ukuran dinamis dapat bertambah dan berkurang ukurannya secara otomatis, biasanya diimplementasikan menggunakan linked list.

Dalam implementasinya, stack memiliki beberapa operasi dasar. Operasi tersebut meliputi push untuk menambahkan elemen ke dalam stack, pop untuk menghapus elemen teratas, peek untuk melihat elemen paling atas tanpa menghapusnya, isEmpty untuk mengecek apakah stack kosong, serta size untuk mengetahui jumlah elemen dalam stack. Operasi-operasi ini menjadi dasar dalam berbagai aplikasi yang menggunakan struktur data stack.

Selain itu, terdapat algoritma dasar dalam operasi push dan pop. Pada operasi push, sistem akan memeriksa apakah stack sudah penuh. Jika belum, maka nilai top akan ditambah satu dan elemen baru dimasukkan ke posisi tersebut. Sedangkan pada operasi pop, sistem akan memeriksa apakah stack kosong. Jika tidak, maka elemen pada posisi top akan diambil, kemudian nilai top dikurangi satu.

2.2 Langkah Kerja

2.2.1 Kode StackArray_2511532024

```
1. public class stackArray_2511532024 {
2.     static final int MAX_2024 = 1000;
3.     int top_2024;
4.     int a_2024[] = new int [MAX_2024];
5.     boolean isEmpty_2024() {
6.         return (top_2024 < 0);
7.     }
8.     stackArray_2511532024(){
9.         top_2024 = -1;
10.    }
11.    boolean push_2024(int x) {
12.        if (top_2024 >= (MAX_2024 - 1)) {
13.            System.out.println("Stack Overflow");
14.            return false;
15.        }
16.        else {
17.            a_2024[++top_2024] = x;
18.            System.out.println(x + " dimasukkan
dalam stack");
19.            return true;
20.        }
21.    }
22.    int pop_2024() {
23.        if (top_2024 < 0) {
24.            System.out.println("Stack Underflow");
25.            return 0;
26.        }
27.        else {
28.            int x = a_2024[top_2024--];
29.            return x;
30.        }
31.    }
32.    int peek_2024() {
33.        if (top_2024 < 0) {
34.            System.out.println("Stack Underflow");
35.            return 0;
36.        }
37.        else {
38.            int x = a_2024[top_2024];
39.            return x;
40.        }
41.    }
42.    void print_2024() {
43.        for (int i = top_2024; i > -1; i--) {
44.            System.out.print(" " + a_2024[i]);
45.        }
46.    }
47. }
```

2.2.2 Penjelasan Kode StackArray_2511532024

1. Deklarasi kelas

```
public class stackArray_2511532024 {
```

Baris ini digunakan untuk mendefinisikan kelas stackArray_2511532024 yang berfungsi sebagai implementasi struktur data stack menggunakan array.

2. Deklarasi konstanta dan variabel

```
static final int MAX_2024 = 1000;
```

```
int top_2024;
```

```
int a_2024[] = new int [MAX_2024];
```

Baris ini mendeklarasikan kapasitas maksimum stack sebesar 1000 elemen melalui konstanta MAX_2024. Variabel top_2024 digunakan untuk menyimpan posisi elemen teratas dalam stack, sedangkan array a_2024 berfungsi sebagai tempat penyimpanan data stack.

3. Method isEmpty

```
boolean isEmpty_2024() {  
    return (top_2024 < 0);  
}
```

Method ini digunakan untuk mengecek apakah stack dalam kondisi kosong. Jika nilai top_2024 kurang dari 0, maka stack dianggap kosong.

4. Konstruktor

```
stackArray_2511532024(){  
    top_2024 = -1;  
}
```

Konstruktor ini digunakan untuk menginisialisasi nilai top_2024 menjadi -1, yang menandakan bahwa stack masih kosong saat pertama kali dibuat.

5. Method push

```
boolean push_2024(int x) {
```

Method ini digunakan untuk menambahkan elemen baru ke dalam stack.

```
if (top_2024 >= (MAX_2024 - 1)) {
```

Baris ini memeriksa apakah stack sudah penuh. Jika top_2024 sudah mencapai kapasitas maksimum, maka terjadi kondisi overflow.

```
a_2024[++top_2024] = x;
```

Jika stack belum penuh, maka nilai top_2024 akan ditambah terlebih dahulu, kemudian elemen x dimasukkan ke posisi tersebut.

6. Method pop

```
int pop_2024() {
```

Method ini digunakan untuk menghapus dan mengambil elemen teratas dari stack.

```
if (top_2024 < 0) {
```

Baris ini mengecek apakah stack kosong. Jika kosong, maka terjadi kondisi underflow.

```
int x = a_2024[top_2024--];
```

Jika tidak kosong, maka elemen pada posisi top_2024 diambil, kemudian nilai top_2024 dikurangi satu.

7. Method peek

```
int peek_2024() {
```

Method ini digunakan untuk melihat elemen teratas tanpa menghapusnya.

```
int x = a_2024[top_2024];
```

Elemen pada posisi top_2024 diambil tanpa mengubah nilai top_2024.

8. Method print

```
void print_2024() {
```

Method ini digunakan untuk menampilkan seluruh isi stack.

```
for (int i = top_2024; i > -1; i--) {  
    System.out.print(" " + a_2024[i]);  
}
```

Perulangan dilakukan dari elemen teratas hingga terbawah, sehingga data ditampilkan sesuai urutan stack (LIFO).

2.2.3 Kode StackArrayDriver_2511532024

```
1. public class stackArrayDriver_2511532024 {  
2.  
3.     public static void main(String[] args) {  
4.         stackArray_2511532024 s = new  
5.         stackArray_2511532024();  
6.         s.push_2024(10);  
7.         s.push_2024(20);  
8.         s.push_2024(30);  
9.         System.out.println(s.pop_2024() + "  
10.         dikeluarkan dari stack");  
11.         System.out.println("Elemen teratas adalah : "  
12.         + s.peek_2024());  
13.         System.out.print("Elemen pada stack      :");  
14.         s.print_2024();  
15.     }  
16. }
```

2.2.4 Penjelasan Kode StackArrayDriver_2511532024

1. Deklarasi kelas

```
public class stackArrayDriver_2511532024 {
```

Baris ini mendefinisikan kelas stackArrayDriver_2511532024 yang berfungsi sebagai program utama untuk menguji implementasi stack berbasis array yang telah dibuat sebelumnya.

2. Method main

```
public static void main(String[] args) {
```

Method ini merupakan titik awal eksekusi program. Semua perintah yang ada di dalamnya akan dijalankan saat program dieksekusi.

3. Pembuatan objek stack

```
stackArray_2511532024 s = new stackArray_2511532024();
```

Baris ini digunakan untuk membuat objek s dari kelas stackArrayDriver_2511532024. Objek ini akan digunakan untuk memanggil berbagai operasi stack seperti push, pop, dan peek.

4. Operasi push

```
s.push_2024(10);  
s.push_2024(20);  
s.push_2024(30);
```

Baris ini digunakan untuk menambahkan elemen ke dalam stack. Nilai 10, 20, dan 30 dimasukkan secara berurutan, sehingga 30 akan menjadi elemen teratas dalam stack.

5. Operasi pop

```
System.out.println(s.pop_2024() + " dikeluarkan dari  
stack");
```

Baris ini digunakan untuk menghapus elemen teratas dari stack. Karena prinsip LIFO, nilai 30 akan dikeluarkan terlebih dahulu dan ditampilkan ke layar.

6. Operasi peek

```
System.out.println("Elemen teratas adalah : " +  
s.peek_2024());
```

Baris ini digunakan untuk melihat elemen teratas stack tanpa menghapusnya. Setelah operasi pop sebelumnya, elemen teratas sekarang adalah 20.

7. Menampilkan isi stack

```
System.out.print("Elemen pada stack  :");  
s.print_2024();
```

Baaris ini digunakan menampilkan seluruh isi stack menggunakan method print_2024(). Data ditampilkan dari atas ke bawah sesuai dengan prinsip stack.

2.2.5 Kode StackPostfix_2511532024

```
1. import java.util.Scanner;  
2. import java.util.Stack;  
3.  
4. public class StackPostfix_2511532024 {  
5.     public static int postfixEvaluate(String expression)  
6.     {  
7.         Stack<Integer> s_2024 = new Stack<Integer>();  
8.         Scanner input_2024 = new Scanner (expression);  
9.         while (input_2024.hasNext()) {  
10.             if (input_2024.hasNextInt()) {  
11.                 // an operand (integer)  
12.                 s_2024.push  
13.                 (input_2024.nextInt());  
14.             } else {  
15.                 // an operator  
16.                 String operator =  
17.                 input_2024.next();  
18.                 int operand2 = s_2024.pop();  
19.                 int operand1 = s_2024.pop();  
20.                 if (operator.equals("+")) {  
21.                     s_2024.push(operand1 +  
22.                     operand2);  
23.                 } else if (operator.equals("-"))  
24.                 {  
25.                     s_2024.push(operand1 -  
26.                     operand2);  
27.                 } else if (operator.equals("*"))  
28.                 {  
29.                     s_2024.push(operand1 *  
30.                     operand2);  
31.                 } else {  
32.                     s_2024.push(operand1 /  
33.                     operand2);  
34.                 }  
35.             }  
36.         }  
37.         input_2024.close();  
38.         return s_2024.pop();  
39.     }  
40.     public static void main(String[] args) {  
41.         System.out.println("hasil postfix= " +  
42.         postfixEvaluate("5 2 4 * + 7 -"));  
43.     }  
44. }
```

2.2.6 Penjelasan Kode StackPostfix_2511532024

1. Import library

```
import java.util.Scanner;  
import java.util.Stack;
```

Baris ini digunakan untuk mengimpor kelas Scanner dan Stack. Scanner digunakan untuk membaca token dari string ekspresi, sedangkan Stack digunakan sebagai struktur data utama untuk proses evaluasi postfix.

2. Deklarasi kelas

```
public class StackPostfix_2511532024 {
```

Baris ini mendefinisikan kelas StackPostfix_2511532024 yang digunakan untuk melakukan evaluasi ekspresi postfix menggunakan struktur data stack.

3. Method evaluasi postfix

```
public static int postfixEvaluate(String expression) {
```

Method ini digunakan untuk menghitung hasil dari ekspresi postfix yang diberikan dalam bentuk string. Method bersifat static sehingga dapat langsung dipanggil tanpa membuat objek.

4. Inisialisasi stack dan scanner

```
Stack<Integer> s_2024 = new Stack<Integer>();  
Scanner input_2024 = new Scanner (expression);
```

Baris ini membuat objek stack untuk menyimpan operand sementara, serta objek Scanner untuk membaca isi string ekspresi secara bertahap.

5. Perulangan membaca token

```
while (input_2024.hasNext()) {
```

Perulangan ini digunakan untuk membaca setiap token (angka atau operator) dalam ekspresi hingga habis.

6. Pengecekan operand

```
if (input_2024.hasNextInt()) {  
    s_2024.push (input_2024.nextInt());  
}
```

Jika token yang dibaca berupa bilangan bulat, maka nilai tersebut dimasukkan ke dalam stack sebagai operand.

7. Pengecekan operator

```
else {  
    String operator = input_2024.next();
```

Jika token bukan angka, maka dianggap sebagai operator seperti +, -, *, atau /.

8. Mengambil operand dari stack

```
int operand2 = s_2024.pop();  
int operand1 = s_2024.pop();
```

Dua nilai diambil dari stack untuk dilakukan operasi. Urutan penting: operand pertama adalah yang lebih lama (operand1), dan operand kedua adalah yang terakhir dimasukkan (operand2).

9. Proses operasi

```
if (operator.equals("+")) {  
    s_2024.push(operand1 + operand2);  
} else if (operator.equals("-")) {  
    s_2024.push(operand1 - operand2);  
} else if (operator.equals("*")) {
```

```
s_2024.push(operand1 * operand2);  
} else {  
    s_2024.push(operand1 / operand2);  
}
```

Bagian ini digunakan untuk melakukan operasi sesuai dengan operator yang ditemukan. Hasil dari operasi kemudian dimasukkan kembali ke dalam stack.

10. Menutup scanner

```
input_2024.close();
```

Baris ini digunakan untuk menutup objek Scanner setelah selesai digunakan.

11. Mengembalikan hasil

```
return s_2024.pop();
```

Setelah semua operasi selesai, hasil akhir akan berada di dalam stack. Nilai tersebut diambil dan dikembalikan sebagai hasil fungsi.

12. Method main

```
public static void main(String[] args) {
```

Method utama yang digunakan untuk menjalankan program.

13. Menampilkan hasil

```
System.out.println("hasil postfix= " + postfixEvaluate("5 2 4  
* + 7 -"));
```

Baris ini memanggil method postfixEvaluate dengan ekspresi postfix “5 2 4 * + 7 -“, kemudian menampilkan hasil perhitungannya.

2.2.7 Kode NilaiMaksimum_2511532024

```
1. import java.util.Stack;
2.
3. public class NilaiMaksimum_2511532024 {
4.
5.     public static int max_2024 (Stack<Integer> s_2024){
6.         Stack<Integer> backup_2024 = new
7.         Stack<Integer>();
8.         int maxValue_2024 = s_2024.pop();
9.         backup_2024.push(maxValue_2024);
10.        while (!s_2024.isEmpty()) {
11.            int next = s_2024.pop();
12.            backup_2024.push(next);
13.            maxValue_2024 = Math.max(maxValue_2024,
14.            next);
15.        }
16.        while (!backup_2024.isEmpty()) {
17.            s_2024.push(backup_2024.pop());
18.        }
19.        return maxValue_2024;
20.    }
21.
22.    public static void main(String[] args) {
23.        Stack<Integer> s_2024 = new Stack<Integer>();
24.        s_2024.push(70);
25.        s_2024.push(12);
26.        s_2024.push(20);
27.        System.out.println("isi stack " + s_2024);
28.        System.out.println("Stack Teratas " +
29.        s_2024.peek());
30.        System.out.println("Nilai maksimum " +
31.        max_2024(s_2024));
32.    }
33. }
```

2.2.8 Penjelasan Kode NilaiMaksimum_2511532024

1. Import library

```
import java.util.Stack;
```

Baris ini digunakan untuk mengimpor kelas Stack dari java collections framework yang akan digunakan sebagai struktur data utama dalam program.

2. Deklarasi kelas

```
public class NilaiMaksimum_2511532024 {
```

Baris ini mendefinisikan kelas NilaiMaksimum_2511532024 yang berfungsi untuk mencari nilai maksimum dalam sebuah stack.

3. Method mencari nilai maksimum

```
public static int max_2024 (Stack<Integer> s_2024){
```

Method ini digunakan untuk mencari nilai maksimum dari elemen-elemen dalam stack. Method bersifat static sehingga dapat dipanggil langsung tanpa membuat objek.

4. Membuat stack backup

```
Stack<Integer> backup_2024 = new Stack<Integer>();
```

Baris ini membuat stack tambahan yang digunakan untuk menyimpan sementara elemen dari stack utama agar data tidak hilang.

5. Inisialisasi nilai maksimum

```
int max_value_2024 = s_2024.pop();  
backup_2024.push(max_value_2024);
```

Elemen pertama diambil dari stack utama dan dijadikan sebagai nilai awal maksimum. Nilai tersebut juga disimpan ke dalam stack cadangan.

6. Perulangan mencari nilai maksimum

```
while (!s_2024.isEmpty()) {
```

Perulangan ini akan berjalan selama stack utama masih memiliki elemen.

```
int next = s_2024.pop();  
backup_2024.push(next);  
max_value_2024 = Math.max(max_value_2024, next);
```

Setiap elemen diambil dari stack utama, disimpan ke stack cadangan, lalu dibandingkan dengan nilai maksimum saat ini menggunakan Math.max() untuk mendapatkan nilai terbesar.

7. Mengembalikan isi stack

```
while (!backup_2024.isEmpty()) {  
    s_2024.push(backup_2024.pop());  
}
```

Setelah semua elemen diproses, isi stack dikembalikan ke kondisi semula dengan memindahkan kembali elemen dari stack cadangan ke stack utama.

8. Mengembalikan nilai maksimum

```
return maxValue_2024;
```

Method mengembalikan nilai maksimum yang telah ditemukan.

9. Method main

```
public static void main(String[] args) {
```

Method utama sebagai titik awal eksekusi program.

10. Inisialisasi dan pengisian stack

```
Stack<Integer> s_2024 = new Stack<Integer>();  
s_2024.push(70);  
s_2024.push(12);  
s_2024.push(20);
```

Baris ini membuat stack dan menambahkan beberapa elemen ke dalamnya, yaitu 70, 12, dan 20.

11. Menampilkan isi stack

```
System.out.println("isi stack " + s_2024);
```

Menampilkan seluruh isi stack.

12. Menampilkan elemen teratas

```
System.out.println("Stack Teratas " + s_2024.peek());
```

Menampilkan elemen paling atas tanpa menghapusnya menggunakan method peek().

13. Menampilkan nilai maksimum

```
System.out.println("Nilai maksimum " +  
max_2024(s_2024));
```

Memanggil method max_2024() untuk mencari nilai terbesar dalam stack, lalu menampilkannya.

2.2.9 Kode SiswaStack_2511532024

```
1. import java.util.ArrayList;  
2.  
3. class Siswa{  
4.     String nama;  
5.     int nim;  
6.     public Siswa (String nama, int nim) {  
7.         this.nama = nama;  
8.         this.nim = nim;  
9.     }  
10.  
11.     @Override  
12.     public String toString() {  
13.         return "Nim: " + nim + ", Nama: " +  
14.         nama;  
15.     }  
16. }  
17. public class SiswaStack_2511532024 {  
18.     private ArrayList<Siswa> stack;  
19.  
20.     public SiswaStack_2511532024() {  
21.         stack = new ArrayList<>();  
22.     }  
23.  
24.     public void push (Siswa mhs) {  
25.         stack.add(mhs);  
26.     }  
27.  
28.     public Siswa pop() {  
29.         if (!isEmpty()) {  
30.             return stack.remove(stack.size()  
31.             - 1);  
32.         }  
33.         return null;  
34.     }  
35.     public Siswa peek() {  
36.         if (!isEmpty()) {
```

```

37.         return stack.remove(stack.size()
- 1);
38.     }
39.     return null;
40. }
41.
42.     public boolean isEmpty() {
43.         return stack.isEmpty();
44.     }
45.
46.     public void tampilkanSiswa() {
47.         for (int i = stack.size() - 1; i >= 0;
i--) {
48.             System.out.println(stack.get(i));
49.         }
50.     }
51.     public static void main (String[] args) {
52.         SiswaStack_2511532024 studentStack =
new SiswaStack_2511532024();
53.
54.         Siswa mhs1 = new Siswa("Ali", 1);
55.         Siswa mhs2 = new Siswa("Boby", 2);
56.         Siswa mhs3 = new Siswa("Charles", 3);
57.         studentStack.push(mhs1);
58.         studentStack.push(mhs2);
59.         studentStack.push(mhs3);
60.
61.         System.out.println("Siswa di dalam
stack:");
62.         studentStack.tampilkanSiswa();
63.
64.         System.out.println("siswa teratas " +
studentStack.peek());
65.         System.out.println("mengeluarkan siswa
teratas dari stack: " + studentStack.pop());
66.         System.out.println("daftar siswa
setelah di pop :");
67.         studentStack.tampilkanSiswa();
68.     }
69. }

```

2.2.10 Penjelasan Kode SiswaStack_2511532024

1. Import library

```
import java.util.ArrayList;
```

Baris ini digunakan untuk mengimpor kelas ArrayList yang akan digunakan sebagai dasar implementasi stack. Berbeda dari stack bawaan, di sini stack dibuat manual menggunakan arraylist.

2. Deklarasi kelas siswa

```
class Siswa{
```

Baris ini mendefinisikan kelas Siswa yang digunakan sebagai tipe data objek yang akan disimpan dalam stack.

3. Atribut kelas siswa

```
String nama;  
int nim;
```

Dua atribut ini digunakan untuk menyimpan data siswa, yaitu nama dan nomor induk mahasiswa (NIM).

4. Konstruktor kelas siswa

```
public Siswa (String nama, int nim) {  
    this.nama = nama;  
    this.nim = nim;  
}
```

Konstruktor ini digunakan untuk menginisialisasi objek siswa dengan nilai nama dan NIM saat objek dibuat.

5. Override method string

```
@Override  
public String toString() {  
    return "Nim: " + nim + ", Nama: " + nama;  
}
```

Method ini digunakan untuk menampilkan data siswa dalam bentuk string yang rapi ketika objek dicetak.

6. Deklarasi kelas stack

```
public class SiswaStack_2511532024 {
```

Baris ini mendefinisikan kelas utama yang berfungsi sebagai implementasi stack untuk menyimpan objek siswa.

7. Deklarasi arraylist sebagai stack

```
private ArrayList<Siswa> stack;
```

Variabel stack digunakan sebagai tempat penyimpanan data siswa dengan konsep stack menggunakan arraylist.

8. Konstruktor stack

```
public SiswaStack_2511532024() {  
    stack = new ArrayList<>();  
}
```

Konstruktor ini menginisialisasi arraylist kosong sebagai stack saat objek dibuat.

9. Method push

```
public void push (Siswa mhs) {  
    stack.add(mhs);  
}
```

Method ini digunakan untuk menambahkan objek siswa ke dalam stack. Data akan ditambahkan dibagian akhir list sebagai elemen teratas.

10. Method pop

```
public Siswa pop() {  
    if (!isEmpty()) {  
        return stack.remove(stack.size() - 1);  
    }  
    return null;  
}
```

Method ini digunakan untuk mengambil dan menghapus elemen teratas dari stack, jika stack kosong, method akan mengembalikan null.

11. Method peek

```
public Siswa peek() {  
    if (!isEmpty()) {  
        return stack.remove(stack.size() - 1);  
    }  
    return null;  
}
```

Method ini seharusnya digunakan untuk melihat elemen teratas tanpa menghapusnya, tetapi pada kode ini justru menggunakan `remove()`, sehingga elemen ikut terhapus. Ini sebenarnya kesalahan logika kecil yang sering terjadi.

12. Method isEmpty

```
public boolean isEmpty() {  
    return stack.isEmpty();  
}
```

Method ini digunakan untuk mengecek apakah stack dalam kondisi kosong atau tidak.

13. Method menampilkan data

```
public void tampilkanSiswa() {  
    for (int i = stack.size() - 1; i >= 0; i--) {  
        System.out.println(stack.get(i));  
    }  
}
```

Method ini digunakan untuk menampilkan seluruh isi stack dari atas ke bawah sesuai konsep LIFO.

14. Method main

```
public static void main (String[] args) {
```

Method utama sebagai titik awal program dijalankan.

15. Pembuatan objek stack

```
SiswaStack_2511532024    studentStack    =    new  
SiswaStack_2511532024();
```

Membuat objek stack untuk menyimpan data siswa.

16. Pembuatan objek siswa

```
Siswa mhs1 = new Siswa("Ali", 1);  
Siswa mhs2 = new Siswa("Boby", 2);  
Siswa mhs3 = new Siswa("Charles", 3);
```

Membuat beberapa objek siswa yang akan dimasukkan ke dalam stack.

17. Operasi push

```
studentStack.push(mhs1);  
studentStack.push(mhs2);  
studentStack.push(mhs3);
```

Menambahkan objek siswa ke dalam stack. Elemen terakhir (Charles) akan berada di posisi teratas.

18. Menampilkan isi stack

```
System.out.println("Siswa di dalam stack:");  
studentStack.tampilkanSiswa();
```

Menampilkan seluruh data siswa yang ada dalam stack.

19. Menampilkan elemen teratas

```
System.out.println("siswa teratas " + studentStack.peek());
```

Menampilkan elemen teratas stack, meskipun dalam implementasi ini elemen tersebut juga ikut terhapus karena kesalahan pada method peek().

20. Operasi pop

```
System.out.println("mengeluarkan siswa teratas dari stack: "
+ studentStack.pop());
```

Menghapus dan menampilkan elemen teratas dari stack.

21. Menampilkan data setelah pop

```
System.out.println("mengeluarkan siswa teratas dari stack: "
+ studentStack.pop());
```

Menampilkan isi stack setelah salah satu elemen dihapus.

2.2.11 Output Kode Program

1. Output Kode Program stackArrayDriver_2511532024

```
<terminated> stackArrayDriver_2511532024 [
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack      : 20 10
```

2. Output Kode Program StackPostfix_2511532024

```
<terminated> StackPostfix_25
hasil postfix= 6
```

3. Output Kode Program NilaiMaksimum_2511532024

```
<terminated> NilaiMaksimum_2511
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

4. Output Kode Program SiswaStack_2511532024

```
<terminated> SiswaStack_2511532024 [Java Application] C:\Users\LENOVO\p21p
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 2, Nama: Bobby
daftar siswa setelah di pop :
Nim: 1, Nama: Ali
```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan struktur data yang bekerja dengan prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Implementasi stack dapat dilakukan menggunakan array, kelas stack bawaan java, maupun menggunakan arraylist.

Melalui berbagai percobaan yang telah dilakukan, seperti operasi dasar (push, pop, peek), evaluasi ekspresi postfix, pencarian nilai maksimum, serta penerapan pada data objek, dapat dipahami bahwa stack memiliki peran penting dalam pengolahan data dan penyelesaian masalah dalam pemrograman.

Dengan demikian, pemahaman terhadap struktur data stack sangat membantu dalam meningkatkan kemampuan logika dan efisiensi dalam pembuatan program.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Stack (Struktur Data)*, Program Studi Informatika, Universitas Andalas, 2026.